# JEUX PARTAGE

Présenté par: Corentin Polard



# CONTEXTE

## Pour qui ?

Amateurs et passionnés de jeux de sociétés

Pour toutes personnes cherchant des partenaires de jeu.

## Pourquoi ?

Une plateforme pour connecter les amateurs et passionnés de jeux de société afin de créer des moments conviviaux autour d’un centre d’intérêt commun.



# SOMMAIRE


1. Fonctionnalités
2. Technologies et outils
3. Analyse et conception
4. Conception graphique
5. Frontend
6. Diagramme de classes et Base de données
7. Backend
8. Sécurité
9. Tests
10. Déploiement
11. Le résultat
12. Améliorations et évolution
13. Conclusion

# FONCTIONNALITÉS

Fonctionnalités principales

Création d’événements

Inscription/Désinscription

Gestion administrateur

Liste avec filtres

Messagerie

# TECHNOLOGIES ET OUTILS


Trello


StarUML


Figma


GitHub


WAMP 3.3.7


9.1.0


PHP 8.3.14


HTML


CSS


JavaScript

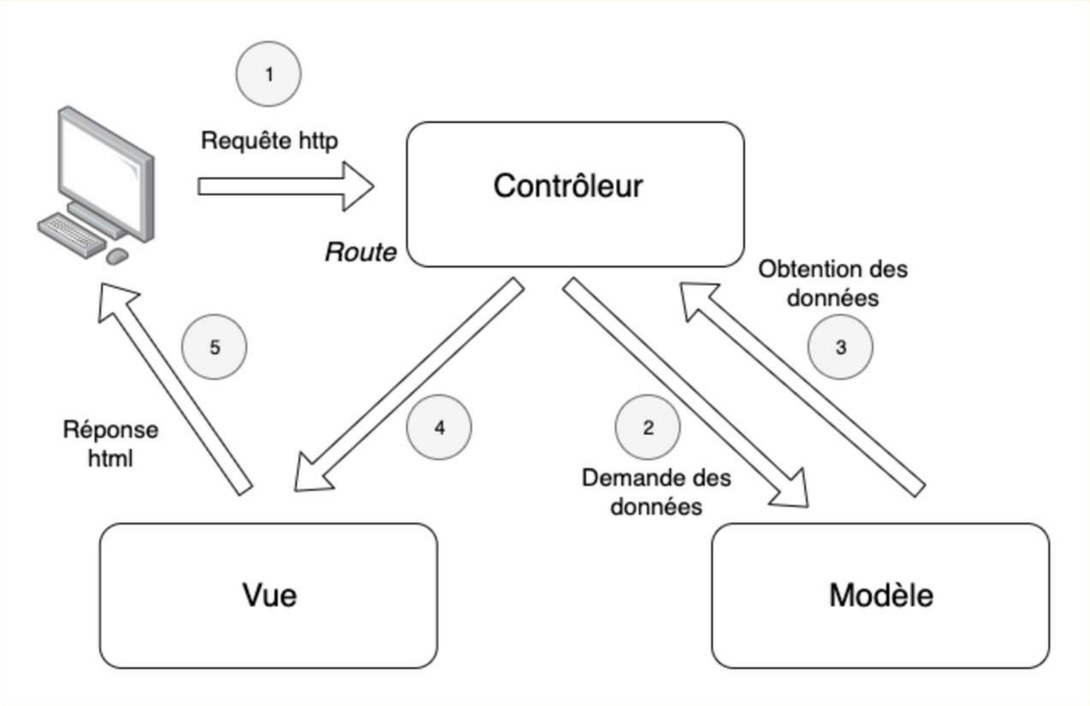
# TECHNOLOGIES ET OUTILS

Symfony 7.3.3

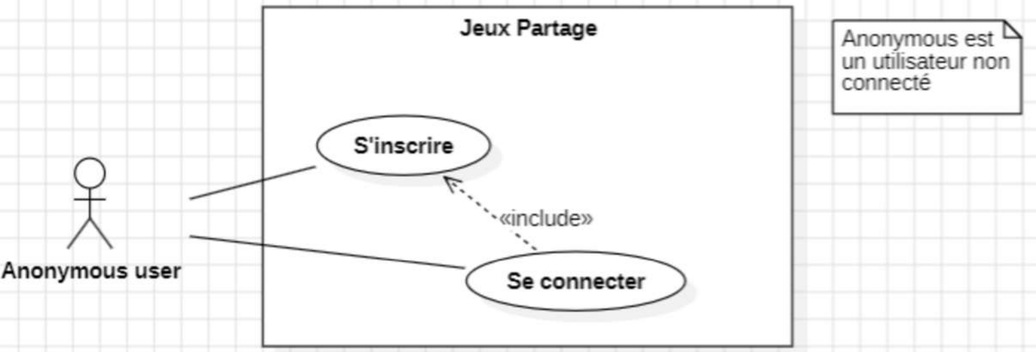
Twig 3.21.1

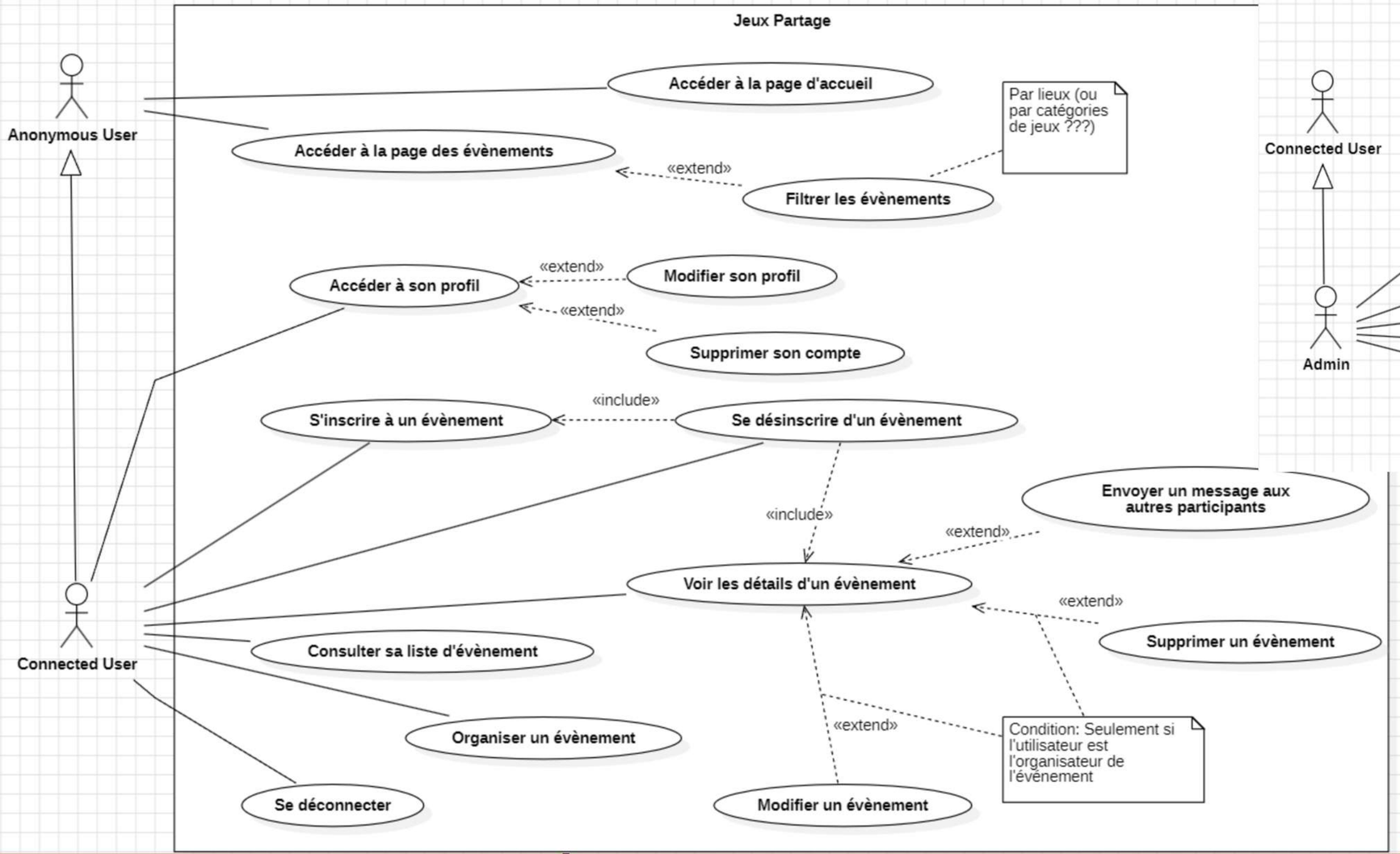
Doctrine 3.5.2

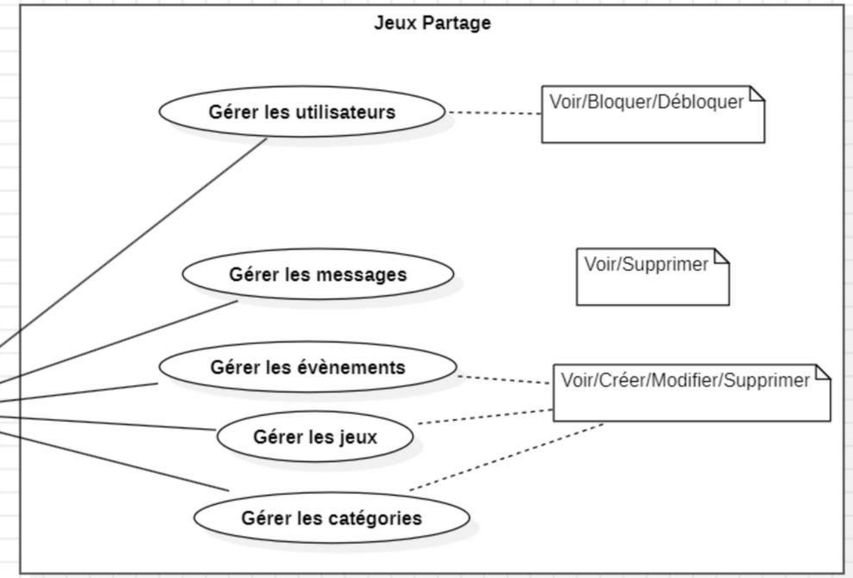
Architecture MVC



# Diagramme de cas d'utilisation



# CONCEPTION GRAPHIQUE

Couleurs

#F9F6E1

#632024

#25344F

#000000

#F9F6E1

#000000

#D5B893

Polices

**DM Serif Display**

Nunito Sans

Logo



# MAQUETTES

LIEN VERS FIGMA

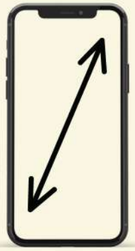
# FRONTEND

Démo vidéo

# FRONTEND

## Comment j’ai rendu mon site responsive ?

**Mobile First**



**Beaucoup de CSS :**

- Display: flex;
- Taille de l’élément selon la taille du parent ou de l’écran
- Changement de disposition facile (espacement, alignement et disposition dynamique) selon l’espace disponible
- Media Queries

# FRONTEND

```twig
{% for game in games %}
<a class="game-link" href="{{ path('app_show_game', {id: game.id}) }}">
    <div class="game-container">
        {% if game.imageFileName %}
        <img class="game-image" src="{{ asset('images/uploads/games/' ~ game.imageFileName) }}"
            alt="Illustration de {{ game.name }}." />
        {% endif %}
        <p class="game-title">{{ game.name }}</p>
    </div>
</a>
{% endfor %}
```

```css
.games-section {
    display: flex;
    flex-flow: row wrap;
    justify-content: center;
    gap: 20px;
    padding: 10px 20px;
}
```

```css
.game-container {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 4px;
    background-color: var(--secondary-background-color);
    box-shadow: rgba(0, 0, 0, 0.35) 0px 5px 15px;
    border-radius: 10px;
}

.game-title {
    flex: 1;
}

.game-container:hover {
    transform: scale(1.05);
    transition: transform 0.3s;
}

.game-container p {
    display: flex;
    flex-direction: column;
    justify-content: center;
    max-width: 150px;
    padding: 10px;
    text-align: center;
}

.game-container img {
    border-radius: 10px 10px 0 0;
}

.game-image {
    max-width: 150px;
    max-height: 150px;
    object-fit: contain;
}
```

# FRONTEND




Mr Jack




Trio




2 pommes 3 pains




The Crew

# FRONTEND

Démo vidéo

# INTERACTIVITÉ

```
const form = document.querySelector("#eventForm");
if (form) {
    form.addEventListener("submit", async function (e) {
        e.preventDefault();

        const street = document.querySelector("#event_address_street");
        const city = document.querySelector("#event_address_city");
        const postcode = document.querySelector("#event_address_postcode");

        const longitude = document.querySelector("#event_address_longitude");
        const latitude = document.querySelector("#event_address_latitude");

        if (street && postcode && city) {
            try {
                const url = `https://data.geopf.fr/geocodage/search?index=address,poi&q=${encodeURIComponent(street.value)}&postcode=${encodeURIComp
                const response = await fetch(url);
                if (!response.ok) {
                    alert("Erreur liée à l'adresse lors de la soumission du formulaire.");
                    return;
                }
                const datas = await response.json();
                if (!datas.features || datas.features.length === 0) {
                    alert("Aucune adresse trouvée !");
                    return;
                }
                const coordinates = [...datas.features[0].geometry.coordinates]; //longitude puis latitude
                longitude.value = coordinates[0];
                latitude.value = coordinates[1];
            } catch (error) {
                console.error(error);
            }
        } else {
            alert("Certains champs d'adresse sont manquants !");
            return;
        }

        form.submit();
```

# INTERACTIVITÉ

Twig

```twig
{% if event.address.latitude and event.address.longitude %}
<div id="map" class="maps" data-address="{{ jsonCoordinates }}">
</div>
{% endif %}
```

JS

```js
import "leaflet/dist/leaflet.css";
import L from "leaflet";
import markerIcon from 'leaflet/dist/images/marker-icon.png';
import markerIcon2x from 'leaflet/dist/images/marker-icon-2x.png';
import markerShadow from 'leaflet/dist/images/marker-shadow.png';

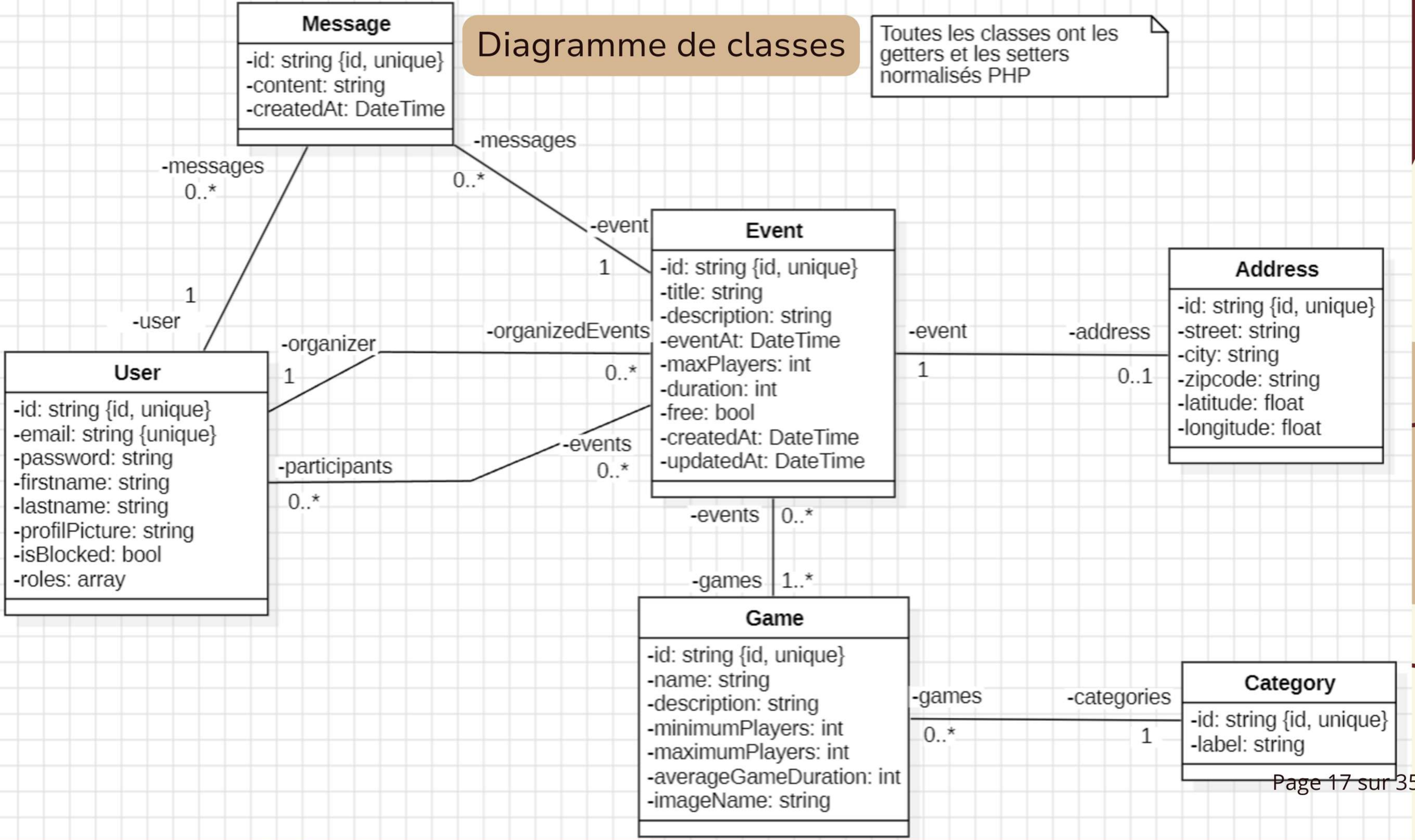
L.Icon.Default.mergeOptions({
    iconUrl: markerIcon,
    iconRetinaUrl: markerIcon2x,
    shadowUrl: markerShadow,
});

let mapContainer = document.querySelector("div[data-address]");
if (mapContainer) {
    const coordinates = JSON.parse(mapContainer.dataset.address).address;

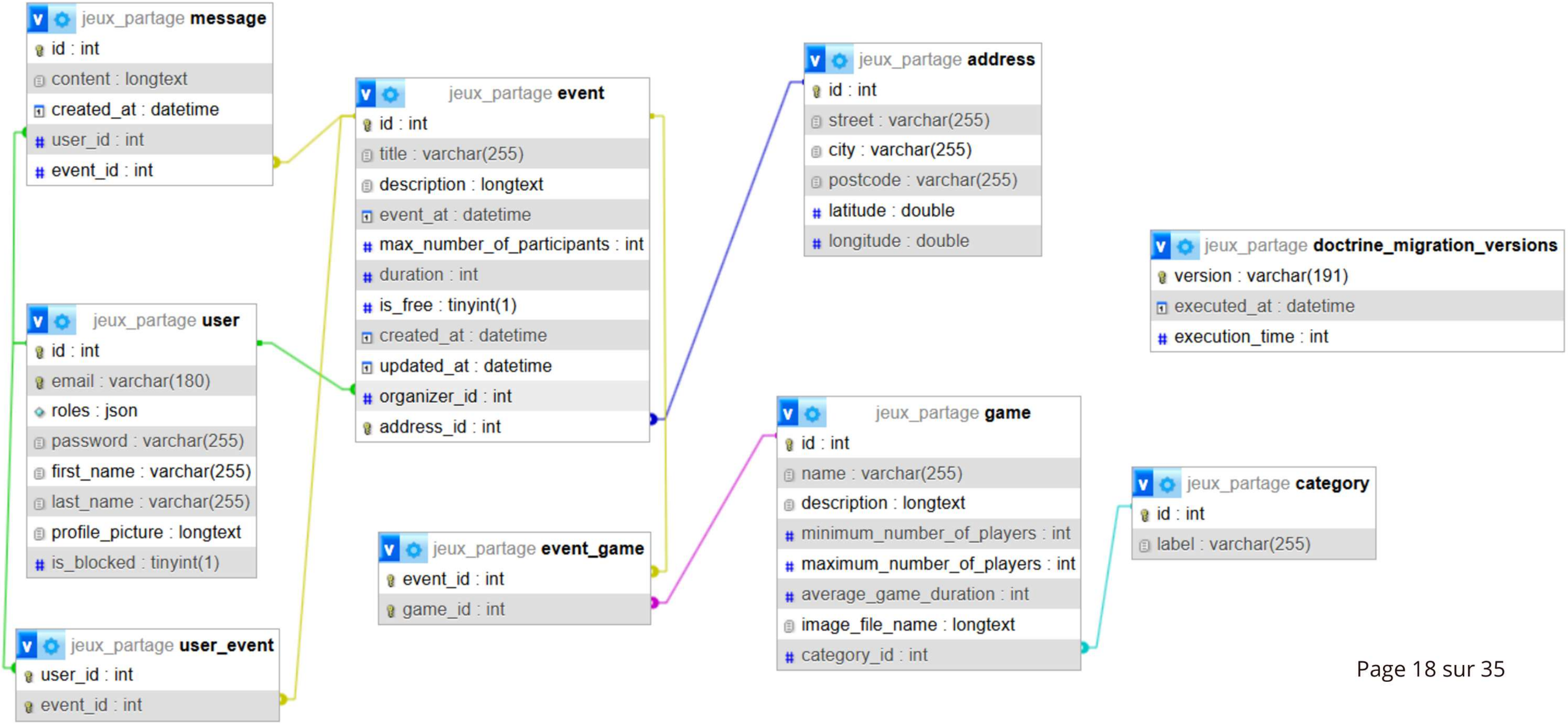
    mapContainer = L.map('map').setView([coordinates.latitude, coordinates.longitude], 16);
    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
        attribution: '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>',
        maxZoom: 19
    }).addTo(mapContainer);

    L.marker([coordinates.latitude, coordinates.longitude]).addTo(mapContainer);
}
```

# Diagramme de classes



# BASE DE DONNÉES



# BASE DE DONNÉES

## Configurer l’accès à la base de données

### Commandes Symfony

- symfony console doctrine:database:create

- symfony console make:entity
- symfony console make:user

- symfony console make:migration
- symfony console doctrine:migrations:migrate

# BACKEND

```php
#[Route('/events')]
final class EventController extends AbstractController
{
    public function __construct(
        private SerializerInterface $serializer
    ) {}

    #[Route('', name: 'app_events')]
    public function index(EventRepository $eventRepository, Request $request): Response
    {…
    }

    #[Route('/show/{id}', name: 'app_show_event', requirements: ['id' => '\d+'])]
    public function showEvent(Event $event, EntityManagerInterface $entityManager, Request $request): Response
    {…
    }

    #[Route('/create', name: 'app_create_event')]
    public function createEvent(EntityManagerInterface $entityManager, Request $request): Response
    {…
    }

    #[Route('/edit/{id}', name: 'app_edit_event')]
    public function editEvent(Event $event, EntityManagerInterface $entityManager, Request $request): Response
    {…
    }

    #[Route('/delete/{id}', name: 'app_delete_event', requirements: ['id' => '\d+'])]
    public function deleteEvent(Event $event, EntityManagerInterface $entityManager): Response
```

# Qu’est-ce qu’il se passe quand l’utilisateur arrive sur la page “Les évènements” ?



# BACKEND

```php
#[Route('', name: 'app_events')]
public function index(EventRepository $eventRepository, Request $request): Respons
{
    $limit = 10;
    $page = $request->query->getInt('page', 1);

    $city = $request->query->get('city');
    $departmentNumber = $request->query->get('departmentNumber');

    $form = $this->createForm(EventsFilterType::class, [
        'city' => $city,
        'departmentNumber' => $departmentNumber
    ]);
    $form->handleRequest($request);

    $filterBy = $form->isSubmitted() && $form->isValid() ? $form->getData() : [
        'city' => $city,
        'departmentNumber' => $departmentNumber
    ];

    if (in_array($filterBy['departmentNumber'], ["2A", "2B"])) {
        $filterBy['departmentNumber'] = 20;
    }
```

# BACKEND

```php
$events = $eventRepository->paginate($page, $limit, "e", new DateTime(), 'eventAt', $filterBy['city'], $filterBy['departmentNumber']);

$maxPage = $events->count() > 0 ? ceil($events->count() / $limit) : 1;
if ($page < 1) {
    return $this->redirectToRoute('app_events', [
        'page' => 1,
        'city' => $filterBy['city'],
        'departmentNumber' => $filterBy['departmentNumber']
    ]);
}
if ($page > $maxPage) {
    return $this->redirectToRoute('app_events', [
        'page' => $maxPage,
        'city' => $filterBy['city'],
        'departmentNumber' => $filterBy['departmentNumber']
    ]);
}

return $this->render('event/events-list.html.twig', [
    'events' => $events,
    'maxPage' => $maxPage,
    'page' => $page,
    'route' => 'app_events',
    'eventForm' => $form->createView(),
]);
```

# BACKEND

```php
abstract class BaseRepository extends ServiceEntityRepository
{
    public function paginate(
        int $page,
        int $limit,
        string $alias,
        ?DateTime $date = null,
        ?string $orderBy = null,
        ?string $city = null,
        ?string $departmentNumber = null
    ): Paginator {
```

# BACKEND

```php
$queryBuilder = $this->createQueryBuilder($alias)
    ->setFirstResult(($page - 1) * $limit)
    ->setMaxResults($limit);

if ($date) {
    $queryBuilder
        ->andWhere("$alias.eventAt >= :date")
        ->setParameter("date", $date);
}

if ($city || $departmentNumber) {
    $queryBuilder
        ->join("$alias.address", "a");
}

if ($city) {
    $queryBuilder
        ->andWhere("LOWER(a.city) like :city")
        ->setParameter("city", "%" . strtolower($city) . "%");
}

if ($departmentNumber) {
    $queryBuilder
        ->andWhere("a.postcode LIKE :departmentNumber")
        ->setParameter("departmentNumber", "$departmentNumber%");
}

if ($orderBy) {
    $queryBuilder->orderBy("$alias.$orderBy", "ASC");
}

return new Paginator(
    $queryBuilder->getQuery()->setHint(
```

# BACKEND

```
SELECT *
FROM event AS e
JOIN address AS a ON e.address_id = a.id
WHERE e.eventAt > NOW()
AND LOWER(a.city) LIKE %(ville)%
AND a.postcode LIKE valeurFormulaire%
LIMIT 10
OFFSET (numeroDePage - 1)*10;
```

# SÉCURITÉ

**Authentification :** système de connexion avec email + mot de passe (haché)

**Gestion des rôles utilisateurs :**
ROLE_USER → utilisateur connecté
ROLE_ADMIN → accès à la gestion complète

**Autorisation :** certaines pages et actions accessibles seulement si connecté ou administrateur

**Formulaires sécurisés :** Symfony intègre la protection CSRF par défaut.

# SÉCURITÉ

```
access_control:
    - { path: ^/admin, roles: ROLE_ADMIN }
    - { path: ^/profile, roles: ROLE_USER }
    - { path: ^/events/, roles: ROLE_USER }
```

# TESTS

```php
public function getFullName(): string
{
    return $this->firstName . " " . $this->lastName;
}
```

```php
public function testUserGetFullName(): void
{
    $client = static::createClient();
    $entityManager = $client->getContainer()->get('doctrine')->getManager();
    $user = $entityManager->getRepository(User::class)->findOneBy([
        'firstName' => 'Jean',
        'lastName' => 'Petit',
    ]);

    $this->assertEquals('Jean Petit', $user->getFullName());
}
```

# TESTS

```
   > loading App\DataFixtures\AppFixtures
PHPUnit 12.3.7 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.3.14
Configuration: C:\Users\coren\Documents\Simplon\final\jeux-partage\phpunit.xml

.......                                                             7 / 7 (100%)

Time: 00:13.160, Memory: 58.00 MB

OK (7 tests, 12 assertions)
```

# DÉPLOIEMENT

Non déployé

# Démo vidéo

# AMÉLIORATIONS ET EVOLUTIONS

## Améliorations

- Validation email et réinitialisation mot de passe
- Modifier son mot de passe
- Auto-complétion pour l’adresse
- Messagerie Temps Réel avec des WebSockets
- Notifications pour messages et événements proches

## Evolutions

- Classement des jeux populaires
- Choix des jeux par vote des participants à l'événement

**Réseau social**

- Liste d'amis / Système de Follow
- Likes sur événements

# CONCLUSION

# MERCI